NANDINI MEHTA

SOFTWARE ENGINEERING PORTFOLIO

mehtanandini.github.io | GitHub | LinkedIn | nandimehta2204@gmail.com | +49 1763 4668 019

Software systems built for reliability, traceability and measurable outcomes.

This portfolio highlights selected work across backend services, full stack applications, data engineering, applied AI, automated testing, CI/CD, monitoring, high volume data workflows and production focused software development.

01 Backend and APIs

Chargeholidays Hotel Booking Platform

Technologies: Go · REST APIs · PostgreSQL · Firebase · Elasticsearch · Kibana · Docker · Kubernetes · Google Cloud

Impact: Professional backend work at ChargeHorizons on systems processing more than 50,000 events per day.

- Developed Go microservices and REST APIs backed by PostgreSQL and Firebase Firestore.
- Integrated Elasticsearch for hotel search, filtering and availability workflows.
- Built Kibana monitoring for logs, service behaviour and operational visibility.
- Worked with Docker, Kubernetes and CI/CD for automated testing and deployment.

02 Commerce

StoreFront E Commerce Platform

Technologies: React.js · Node.js · PHP · MySQL · JavaScript · MVC

Impact: Built a complete commerce application spanning user flows, backend logic, database operations and testing.

- Implemented a full stack e commerce application with React and Node.js plus a PHP backend.
- Connected product and user workflows to a MySQL data layer.
- Used MVC structure, prototyping and testing cycles to keep application logic maintainable.
- Worked across frontend, backend and database concerns instead of treating them as isolated layers.

03 Testing and Quality

Python Web Applications and Automated Validation

Technologies: Python · Django · Flask · PostgreSQL · Pytest · Docker

Impact: Maintained 5 plus applications serving around 2,000 daily users and reduced the QA cycle by 25 percent.

- Built and maintained Python web applications and REST APIs with Django, Flask and PostgreSQL.
- Automated validation using Pytest and repeatable test workflows.
- Improved maintainability through reusable backend components and database integration.
- Used Docker and Git based development practices to support consistent engineering workflows.

RELIABILITY AND SCALE

04 Monitoring

Real Time Vehicle Fault and Emission Anomaly Detection Platform

Technologies: Python · FastAPI · PostgreSQL · React · WebSocket · Monitoring

Impact: Built an end to end telemetry system with structured alerts, operational KPIs and root cause visibility.

- Streamed and validated vehicle telemetry through backend services and PostgreSQL.
- Built live dashboards with React and WebSocket updates.
- Designed alerts, trends and operational indicators for abnormal system behaviour.
- Structured validation and monitoring so failures could be detected and investigated consistently.

05 Large Data

Euro 7 Automotive Data and ML Pipeline

Technologies: Python · Pandas · NumPy · scikit learn · Autoencoders · PCA · UMAP

Impact: Processed more than 25 GB and approximately 90 million OBD and CAN time series records.

- Built modular processing steps for cleaning, transformation, feature preparation and validation.
- Developed anomaly detection and clustering workflows for noisy real world engineering data.
- Created repeatable model evaluation and result validation procedures.
- Produced traceable technical findings and test specifications for engineering stakeholders.

06 API and Data Integrity

ClaimVision AI Multimodal Insurance Claim Analyzer

Technologies: Python · FastAPI · PostgreSQL · OCR · Structured Extraction · Validation

Impact: Designed a modular processing system where multiple input types produce consistent structured outputs.

- Combined claim documents, repair invoices and vehicle damage images in one analysis workflow.
- Added extracted values, consistency checks and risk indicators with validation across inputs.
- Separated backend processing components so they could be tested, debugged and evolved independently.
- Focused on data contracts, traceability and reliable API ready outputs.

07 Data Processing

Document Classification and Extraction

Technologies: Python · FastAPI · Tesseract · Hugging Face · NLP · Structured Outputs

Impact: Built a reusable document ingestion and extraction workflow with validation at the API boundary.

- Processed PDF and image documents through OCR, classification and field extraction steps.
- Added FastAPI interfaces with input validation and structured response formats.
- Designed reusable processing components for invoices, receipts and statements.
- Used repeatable test inputs to debug extraction quality and data consistency.

AI-NATIVE FULL STACK

08 Document Intelligence and Agent-Ready Workflows

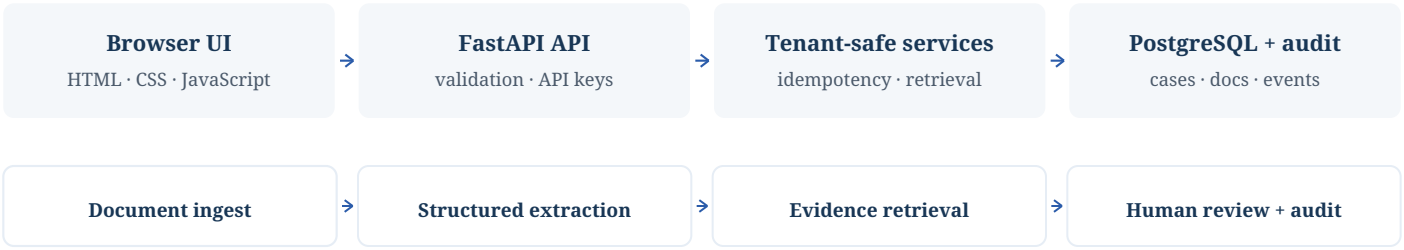
CaseLedger AI

Technologies: Python · FastAPI · SQLAlchemy · PostgreSQL · HTML/CSS/JavaScript · Pytest · Docker · GitHub Actions

Impact: Built a tenant-isolated full-stack document workflow for regulated case files with repeatable ingestion, evidence-backed retrieval, human review and auditability.

- Designed tenant-scoped case and document models with cross-tenant protection, API-key access and idempotent imports for safe repeat processing.
- Built structured extraction and grounded question answering that returns supporting evidence and explicitly flags outputs for human review.
- Added audit trails, input validation, API tests and deterministic local processing so sensitive demo data does not require an external LLM call by default.
- Implemented a lightweight browser UI and containerized PostgreSQL deployment, with GitHub Actions CI for repeatable delivery and regression checks.

ARCHITECTURE AT A GLANCE



Engineering focus

Tenant isolation · idempotency · structured outputs · evidence/citations · human review · auditability · automated tests · Docker/CI

09 **Enterprise AI Delivery and Governance**

Enterprise AI Delivery Workbench

Technologies: Python · FastAPI · React · TypeScript · ChromaDB · SQLAlchemy · Docker Compose

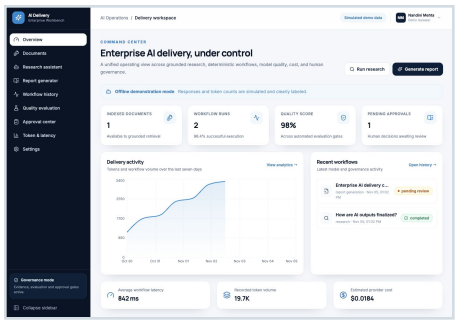
Impact: Built an end-to-end AI delivery platform that keeps enterprise research grounded, measurable and accountable from ingestion through human approval.

- Ingests PDF, DOCX, TXT and Markdown with isolated vector retrieval and verified citations.
- Generates schema-validated reports, quality scores and deterministic workflow replays.
- Tracks tokens, latency, cost and provider health, then records approval decisions in an audit trail.
- Runs locally in a deterministic demo mode and supports OpenAI-compatible and Ollama providers.

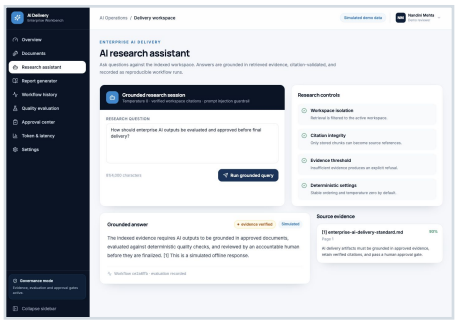
PLATFORM WORKFLOW

INGEST documents > **RETRIEVE** evidence > **GENERATE** reports > **EVALUATE** quality > **APPROVE** audit

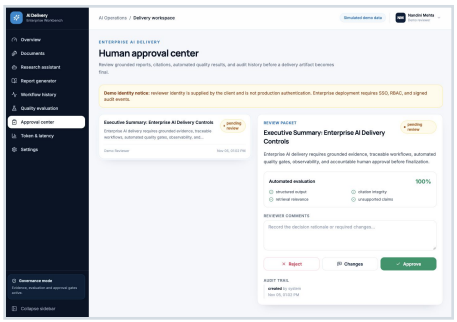
WORKING PRODUCT



Command center



Grounded research



Human approval

Engineering focus

Grounded RAG · structured outputs · evaluation · observability · deterministic replay · human governance

Project links

Interactive portfolio case study

GitHub source repository

Independent portfolio project · Built to demonstrate production-minded AI delivery with traceable evidence, measurable quality and accountable review.

CORE ENGINEERING CAPABILITIES

Software engineering	Go, Python, Java and PHP experience across APIs, microservices, web backends and full stack applications.
Data and databases	Professional PostgreSQL experience plus MySQL, Oracle, Firebase, Elasticsearch and large scale data processing.
Automated testing	Unit and integration testing plus Pytest automation that reduced a QA cycle by 25 percent.
Scalable systems	Services processing more than 50,000 events per day plus a data pipeline over approximately 90 million records.
Messaging and events	Worked with Google Cloud Pub/Sub, MQTT and event driven application components.
Reliability and observability	Elasticsearch, Kibana, Grafana, monitoring, debugging, alerts and service health investigation.
Cloud delivery	Docker, Kubernetes, Google Cloud, Git based workflows and automated CI/CD delivery.
Product development	Built ecommerce, booking, document intelligence, AI and real time monitoring applications across the stack.
AI and agentic systems	LLM APIs, RAG, LangGraph, Hugging Face, multimodal AI, OCR, NLP, semantic search, vector databases and model evaluation.

ADDITIONAL ENGINEERING PROJECT

Wildlife Protecting System | Bachelor Thesis

Built an embedded railway safety system using Java, Arduino and PIR sensors with MQTT and HTTP based alerting. The project strengthened my foundations in event driven communication, real time monitoring and system reliability.

PROFESSIONAL OUTCOMES

50,000+

events per day supported by backend and cloud services

90 million

automotive records processed in a large data pipeline

2,000

daily users across Python web applications

25 percent

reduction in QA cycle through Pytest automation

ENGINEERING STRENGTHS

- Design APIs and software components around clear data contracts and traceable behaviour.
- Treat automated testing, monitoring and CI/CD as part of feature delivery from the beginning.
- Use databases, validation and observability to protect data consistency and diagnose failures quickly.
- Combine backend, full stack, data, AI and agentic systems experience with event driven cloud development.